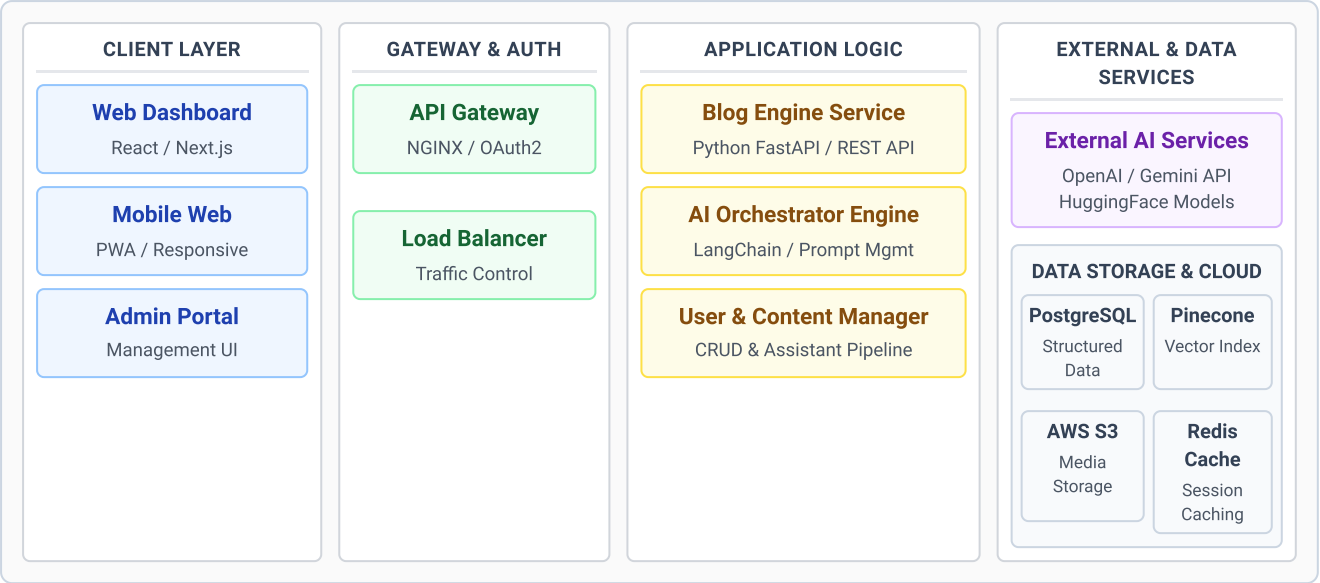
PROJECT DESIGN PHASE-II

Technology Stack & Technical Architecture Document

DATE	PROJECT ID	PROJECT NAME	MAXIMUM MARKS
29 September 2026	SWTID-2026-2363	AI BlogNest	4 Marks

TECHNICAL ARCHITECTURE - AI BLOGNEST SYSTEM



SYSTEM ARCHITECTURE DESIGN GUIDELINES MET

- **Application Logic:** Modular microservices using FastAPI and LangChain orchestration.
- **Infrastructural Demarcation:** Hybrid setup with local Docker containers & AWS/Cloud infrastructure.
- **External Interfaces:** Managed third-party AI APIs (OpenAI / Google Gemini) and Cloud Storage APIs.
- **Data Storage:** PostgreSQL for relational data, Pinecone for vector indexing, and Redis for caching.
- **ML Model Interface:** Direct integration with Generative LLMs for automated content generation and SEO tuning.

TABLE 1: COMPONENTS & TECHNOLOGIES

S.NO	COMPONENT	DESCRIPTION	TECHNOLOGY
1.	**User Interface**	Interactive web interface for bloggers and readers, providing real-time AI generation controls, rich text editor, and responsive dashboard.	**React.js / Next.js, Tailwind CSS, TypeScript**
2.	**Application Logic - 1 (Core Web Server)**	Core application engine managing user authentication, blog posts CRUD, comment moderation, and web routing logic.	**Python / FastAPI (Asynchronous Web Framework)**
3.	**Application Logic - 2 (AI Orchestrator)**	Engine responsible for prompt engineering, workflow chaining, context retrieval, and structured response handling.	**LangChain / LlamaIndex Framework**
4.	**Application Logic - 3 (Background Tasks)**	Asynchronous processing queue for automated SEO optimization, image generation requests, and email notifications.	**Celery with Redis Broker**
5.	**Database (Relational)**	Stores relational data including user profiles, blog metadata, access roles, comments, and transaction logs.	**PostgreSQL Database**
6.	**Vector Database (AI Index)**	Stores vector embeddings of blog content to enable semantic search, context retrieval, and AI content recommendation.	**Pinecone / Qdrant Cloud Vector DB**
7.	**File & Media Storage**	Cloud file storage solution for user avatars, blog cover images, generated assets, and system backups.	**AWS S3 / Cloudflare R2 Bucket**
8.	**External API - 1 (Generative LLM)**	Third-party Large Language Models utilized for auto-generating articles, summaries, headlines, and tone adjustments.	**OpenAI GPT-4o API / Google Gemini API**
9.	**External API - 2 (AI Image Gen)**	Third-party AI service used to automatically generate contextual featured images for published blog posts.	**DALL-E 3 API / Midjourney API**
10.	**Machine Learning Model**	Custom fine-tuned NLP model for spam filtering, sentiment analysis of comments, and automated tag generation.	**HuggingFace Transformer Models / Scikit-learn**
11.	**Infrastructure (Server / Cloud)**	Containerized deployment architecture. • Local Environment: Docker Desktop & Docker Compose. • Cloud Configuration: Kubernetes Cluster on AWS EKS / AWS EC2 instance nodes.	**Docker, Kubernetes, AWS Cloud Infrastructure**

TABLE 2: APPLICATION CHARACTERISTICS (PART 1)

S.NO	CHARACTERISTICS	DESCRIPTION	TECHNOLOGY
1.	**Open-Source Frameworks**	Leverages robust open-source tools to avoid vendor lock-in, ensure rapid development, and maintain high code quality across client and server layers.	**React.js, Next.js, FastAPI, LangChain, PostgreSQL, Redis**

TABLE 2: APPLICATION CHARACTERISTICS (PART 2)

S.NO	CHARACTERISTICS	DESCRIPTION	TECHNOLOGY
2.	**Security Implementations**	Enforces end-to-end data encryption (data at rest and in transit), Role-Based Access Control (RBAC), token authentication, and protection against OWASP top 10 vulnerabilities.	**OAuth2 / JWT Tokens, SHA-256 Hashing, HTTPS / TLS 1.3, AWS WAF, OWASP Security Headers**
3.	**Scalable Architecture**	Designed using a decoupled microservices architecture. Backend application logic and heavy AI processing run independently, enabling horizontal scaling under peak loads.	**Microservices, Kubernetes Horizontal Pod Autoscaler (HPA), Celery Workers**
4.	**Availability**	Ensures 99.9% application uptime by distributing cloud hosting across multiple availability zones with automated health checks and instant failover mechanisms.	**AWS Application Load Balancer (ALB), Multi-AZ PostgreSQL RDS, Health Monitoring**
5.	**Performance**	Optimized for fast page load times and rapid LLM streaming responses. Implements in-memory caching for frequent queries and global CDN for static asset distribution.	**Redis Cache, Cloudflare CDN, Asynchronous Server Gateway Interface (ASGI)**

ARCHITECTURE REFERENCES & STANDARDS

• **C4 Model for Software Architecture:** https://c4model.com/ — Used as a structural baseline for system context and container diagramming.

• **IBM Cloud Architecture Center:** https://www.ibm.com/cloud/architecture — Reference pattern for cloud-native microservice integration.

• **AWS Architecture Center:** https://aws.amazon.com/architecture — Reference design for high-availability vector DB and web application hosting.

• **Technical Architecture Best Practices:** Guidelines on Drawing Useful Architecture Diagrams.

DOCUMENT END - PROJECT DESIGN PHASE-II SUBMISSION

AI BLOGNEST SYSTEM ARCHITECTURE & TECHNOLOGY SELECTION MATRIX